Thread MergeSort Algorithm

2022041069
이인수
운영체제-01
Prof: Heeseung Jo

1. Overview

본 프로그램은 POSIX Thread(pthread)를 활용하여 정수 데이터를 효율적으로 정렬하는 멀티스레드 병합정렬(Multi-threaded Merge Sort)프로그램이다.

2. 개발환경 및 실행방법

2.1 Development Environment

OS: macOS(Apple Silicon M4)
Compiler: GCC(Apple clang version 17.0.0 (clang-1700.6.4.2))
Editor: Neovim

2.2 컴파일 및 실행

```
gcc -O3 -o mergesort mergesort.c -pthread

# 도움말 출력
./mergesort --help

#실행방법
./mergesort <input file> <number of threads>

# 실행 예시
./mergesort data.arr 8 | cat
```

본 프로그램은 MacOS(M4), Ubuntu 22.04(VM 4core), wsl 2.6.3.0 에서 개발-검증했다.

실행 예시(계속)

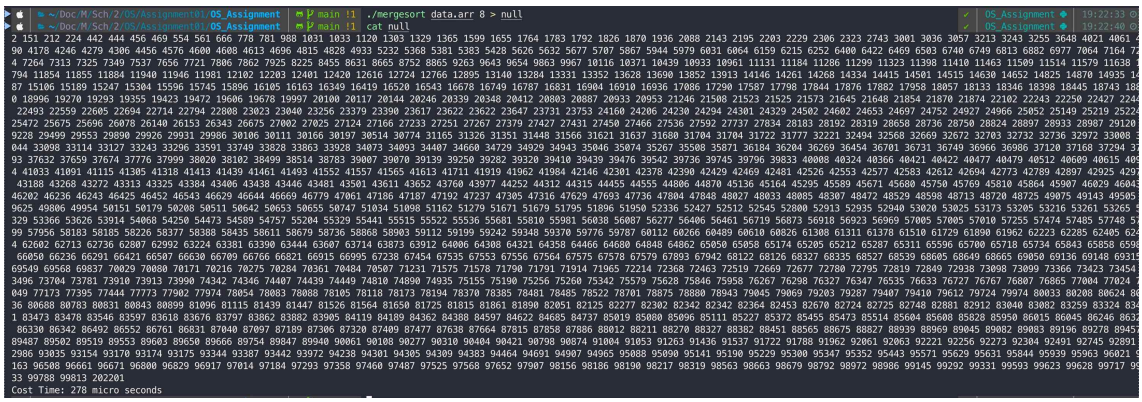


그림 1 파이프를 통해 터미널 오버헤드를 줄일 수 있다.

3. Structure and Algorithm

3.1 process flow

1. Load data: 입력파일로부터 정수를 읽어 1차원 배열 arr에 저장한다.
2. Make thread: 입력받은 스레드 숫자만큼 pthread를 생성한다.
3. Multi-threaded sort: 각 스레드는 지정된 범위에서 merge sort를 수행한다.
4. Synchronization: pthread_join을 call해서 스레드가 종료될 때까지 대기한다.
5. Final Merge: 메인 스레드가 정렬된 N개의 서브셋을 순차적으로 합병한다.
6. Print: 정렬된 출력 결과와 마이크로초단위의 수행시간을 출력한다.

3.2 핵심 함수

1. threadMergeSort: pthread_create호출 시 각 스레드가 가장 먼저 실행하는 함수. 담당 영역에 mergeSort를 호출함

2. mergeSort: 재귀적 병합 정렬함수. 32개 이하면 insertion sort를 수행한다.

3. merge: 이미 정렬된 두 개의 부분 배열을 하나로 합친다.

4. fast_itoa: 출력 최적화 함수. 정수형 데이터를 문자열로 변환해 fwrite한 번으로 출력할 수 있게 버퍼에 문자열을 채운다.

5. main: mmap을 통해 메모리에서 파일을 읽고, N개 만큼 스레드로 나누어 스레드를 만들고, 위의 함수들을 호출한다.

4. 소스코드

happylife10201020 / OS_ThreadMergeSort

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insi

OS_ThreadMergeSort / mergesort.c

LEE and LEE input에 문자열 입으면 에러 8e6ccf8 · yesterday

374 lines (31

Code Blame Raw

```c
/* OS Assignment: Multi-threaded Merge Sort
 * professor: 조희승
 * std num: 2022041069
 * name: 이인수
 *
 * Development Environment
 *  -os: macOS (Apple Silicon M4)
 *  -Editor: Neovim (nvim)
 *  -Compiler: GCC (Apple clang/gcc)
 *
 * Excution Environment
 *  - Verified to run correctly on MacOS (Apple Silicon M4)
 *  - Also tested and verified on Ubuntu 64-bit env (VM)
 *
 * Comilation & Excution
 *  1. gcc -O3 -o mergesort mergesort.c -pthread
 *  2. ./mergesort <input_file> <thread_count>
 *  (ex. ./mergesort data.arr 10
 *
 * Explanation
 *  pthread를 사용해 N개의 스레드 수 만큼 병렬 합병 정렬을 수행한다.
 * */

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<pthread.h>
#include<sys/time.h>
#include<unistd.h>
#include<fcntl.h>
#include<sys/mman.h>
#include<sys/stat.h>
#include<stdbool.h>

#define THRESHOLD 32 //32개 이하 삽입정

typedef struct{
    int* arr ;
    int* temp ;
    int left ;
    int right ;
}ThreadData ;

void merge(int*, int, int, int, int*) ;
void mergeSort(int*, int, int, int*) ;
void* threadMergeSort(void* ) ;
void insertionSort(int[], int, int) ;
```

```c

void* fast_itoa(int, char*) ;


/**
 * brief        Multi-Thread방식으로 MergeSort를 구현
 * *@param  crgv[1] : string    input file
 * @param   crgv[2] : natural int   the number of thread
 */
int main(int argc, char* argv[]) {
    //================================ start time count
    struct timeval startTime, endTime ;
    gettimeofday(&startTime, NULL) ;


    //================================ 입력값 검증
    if(argc != 3){
        if(argc == 2 && strcmp(argv[1], "--help") == 0){ // --help 구
            printf(
                    "\tUsage: %s <file> <N>\n"
                    "\t <file> : path to input file containing natural integers\n"
                    "\t <N> : number of threads\n"
                    "\n\tExample: %s input.txt 4\n",
                    argv[0], argv[0]
                  ) ;
        } else {
            fprintf(stderr,
                    "\tUsage Error: %s <file> <N> \n"
                    "\tTry '%s --help' for more information. \n" ,
                    argv[0], argv[0]
                   ) ;
        }
        return 1 ;
    } else {
    if(atoi(argv[2]) <= 0) {
        fprintf(stderr, "N should bigger than 0\n") ;
        return 1 ;
    }
    }


    //================================ open files현
    /*char* fileName = argv[1];
    int N = atoi(argv[2]);

    FILE *fp ;
    fp = fopen(fileName, "r") ;

    if(fp == NULL) {
        fprintf(stderr, "Failed to open file %s\n", fileName) ;
        return 1 ;
    }
    if(N == 0) {
        fprintf(stderr, "N cannot be 0\n") ;
        return 싱1 ;
    }

    //================================ 배열 만들기
    int capacity = 10 ;
    int* arr = malloc(sizeof(int) * capacity) ;
    int count = 0 ;
    char* line = NULL ;
    size_t len = 0 ;
```

```
111
112         while(getline(&line, &len, fp) != -1) {
113             char* ptr = line ;
114             char* end ;
115             while(1) {
116                 long num = strtol(ptr, &end, 10) ;
117                 if(ptr == end) break ;
118                 if(count == capacity) {
119                     capacity *= 2 ;
120                     arr = realloc(arr, capacity *sizeof(int)) ;
121                 }
122                 arr[count++] = (int)num ;
123                 ptr = end ;
124             }
125         }*/
126
127
128         //================================ mmap 사용한 고속 입출력
129
130         int N = atoi(argv[2]) ;
131         int fd = open(argv[1], O_RDONLY) ;
132
133         if(fd < 0) {
134             perror("Failed to open file\n");
135             return 1 ;
136         }
137
138         struct stat sb ;
139         fstat(fd, &sb) ;
140         char* file_addr = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0) ;
141         if(file_addr == MAP_FAILED) {
142             perror("mmap failed") ;
143             return 1 ;
144         }
145
146         // 숫자 개수 파악
147         int* arr= malloc(sizeof(int) * (sb.st_size / 2)) ;
148         size_t count = 0 ;
149         char* p = file_addr ;
150         char* end_p = file_addr + sb.st_size ;
151
152         //문자로 파
153         bool error_flag = 0 ;
154
155         while( p< end_p) {
156             if(*p == ' ' || *p == '\n' || *p == '\t') {
157                 p++ ;
158                 continue ;
159             }
160             if((*p >= '0' && *p <= '9') || *p == '-') {
161                 int sign = 1 ;
162                 if(*p == '-') {
163                     sign = -1;
164                     p++;
165
166                     if(!(p < end_p && *p >= '0' && *p <= '9')) {
167                         error_flag = 1 ;
168                         break ;
169                     }
170                 }
171                 int num = 0 ;
172                 while(p < end_p && *p >= '0' && *p <= '9') {
173                     num = num * 10 + (*p - '0') ;
```

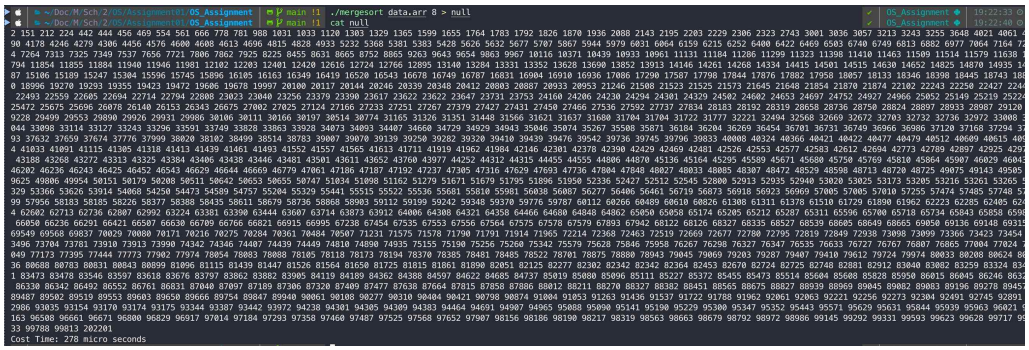
```
                p++ ;
            }
            arr[count++] = num * sign;
        } else {
            error_flag = 1 ;
            break ;
        }
    }

    if(error_flag) {
        fprintf(stderr, "Error: input file contains invalid character\n") ;
        munmap(file_addr, sb.st_size) ;
        close(fd) ;
        free(arr) ;
        return 1 ;
    }

    munmap(file_addr, sb.st_size) ;
    close(fd) ;

    if(count == 0) {
        free(arr) ;
        return 0 ;
    }


    //================================ make thread

    if(N > count) N = count ; // N 이 count보다 클 때
    int chunk = count / N ;
    pthread_t threads[N] ;
    ThreadData tdata[N] ;

    int* global_temp = malloc(sizeof(int) * count) ; // 전체 공유 버퍼


    for(int i = 0; i<N; i++) {
        tdata[i].arr = arr ;
        tdata[i].temp = global_temp ;
        tdata[i].left = i * chunk ;

        if(i == N - 1)
            tdata[i].right = count - 1 ;
        else
            tdata[i].right = (i + 1) * chunk - 1 ;

        //int size = tdata[i].right - tdata[i].left + 1 ;
        //tdata[i].temp = malloc(sizeof(int) * size) ;
        pthread_create(&threads[i], NULL, threadMergeSort, &tdata[i]) ;
    }

    //================================ join
    for(int i = 0; i<N; i++ ) {
        pthread_join(threads[i], NULL) ;
        // free(tdata[i].temp) ; // 정렬 이후 전부 free
    }


    //================================ final merge
    int step = chunk ;
    while(step < count) {
        for(int i = 0; i<count; i+= 2* step) {
            // int left = i ;
```

```
237                 int mid = i + step - 1 ;
238                 int right = i + 2 * step - 1 ;
239                 if(mid >= count) continue ;
240                 if(right >= count) right = count - 1 ;
241                 merge(arr, i, mid, right, global_temp) ;
242             }
243             step *= 2 ;
244         }
245         //================================ output
246         /*
247         printf("%d", arr[0]) ;
248         for(int i = 1; i < count; i++) {
249             printf(" %d", arr[i]) ;
250         }
251         printf("\n") ;
252         */
253 
254         //buffered write
255 
256         char* out_buf = malloc(count * 12) ; //숫자당 최대 11자 + 공백
257         char* out_p = out_buf ;
258         for(int i = 0; i<count; i++) {
259             out_p = fast_itoa(arr[i], out_p) ;
260             *out_p++ = (i == count - 1) ? '\n' : ' ' ;
261         }
262         fwrite(out_buf, 1, out_p - out_buf, stdout) ;
263 
264         //================================ free and print time
265         //free(arr) ;
266         //free(line) ;
267         //fclose(fp) ;
268         free(global_temp) ;
269         free(out_buf) ;
270         //free(temp) ;
271         free(arr) ;
272 
273 
274         gettimeofday(&endTime, NULL) ;
275         long long diff = (endTime.tv_sec - startTime.tv_sec) * 1000000LL + (endTime.tv_usec - startTime.tv_usec) ;
276 
277         printf("Cost Time: %lld micro seconds\n", diff) ; //최종 소요시간 출력
278 
279         return 0 ;
280     }
281 
282 
283     /**
284      * @breif       두 개의 정렬된 부분 배열을 하나의 정렬된 배열로 합침
285      * * @param arr 왼쪽 데이터 배열
286      * @param   left    왼쪽 부분 배열의 시작 위치
287      * @param   mid 왼쪽 부분 배열의 끝 위치
288      * @param   right   오른쪽 부분 배열의 끝 위치
289      * @param   temp    합병 과정에서 사용할 임시 공유 버퍼
290      * @return  void
291      */
292     void merge(int* arr, int left, int mid, int right, int* temp) {
293         int i = left, j = mid + 1, k = left ;
294 
295         while(i <= mid && j <= right) {
296             if(arr[i] < arr[j]) temp[k++] = arr[i++] ;
297             else temp[k++] = arr[j++] ;
298         }
299 
```

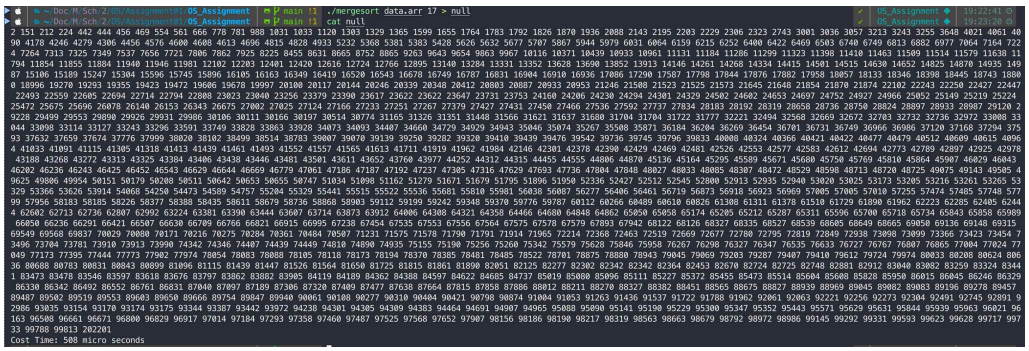
```
    while(i<= mid) temp[k++] = arr[i++] ;
    while(j <= right) temp[k++] = arr[j++] ;

    for(int t = left; t <= right; t++)
        arr[t] = temp[t] ;
}



/**
 * @brief       재귀적으로 배열을 분할하고 합병 정렬을 수행
 ** @param  arr     정렬할 정수 배열의 포인터
 * @param   left    정렬 부분의 시작 인덱스
 * @param   right   정렬 부분의 끝 인덱스
 * @param   temp    합병 과정에서 사용할 임시 공유 버퍼 : malloc을 한 번만 하기 위해
 * @return  void
 */
void mergeSort(int* arr, int left, int right, int* temp) {
    // if(left >= right) return ;

    if(right - left + 1 <= THRESHOLD) {
        insertionSort(arr, left, right) ;
        return ;
    }
    int mid = (left + right) / 2 ;

    mergeSort(arr, left, mid, temp) ;
    mergeSort(arr, mid+1, right, temp) ;
    merge(arr, left, mid, right, temp) ;
}

void* threadMergeSort(void* args) {
    ThreadData* data = (ThreadData*)args ;
    mergeSort(data->arr, data->left, data->right, data->temp) ;
    return NULL ;
}

void insertionSort(int arr[], int left, int right) {
    for(int i = left + 1; i<= right; i++) {
        int key = arr[i] ;
        int j= i - 1 ;
        while(j >= left && arr[j] > key) {
            arr[j + 1] = arr[j] ;
            j-- ;
        }
        arr[j + 1] = key ;
    }
}

/**
 * @brief       입출력 속도 향상을 위해 정수값을 문자열로 변환
 * * @param val 변환할 정수
 * @param   buf 문자열이 저장될 버퍼의 현재 위치 포인터
 * @return void*    다음 숫자가 써질 버퍼의 위치 포인터 변환
 */
void* fast_itoa(int val, char* buf) {
    if (val == 0) { *buf++ = '0'; return buf; }
    unsigned int uval; // INT_MIN 절대값처리하면 오버플로우남
    if (val < 0) {
        *buf++ = '-';
        uval = (unsigned int)-(val);
    } else {
        uval = (unsigned int)val;
```

```
    }
    char temp[12];
    int i = 0;
    while (uval > 0) {
        temp[i++] = (uval % 10) + '0';
        uval /= 10;
    }
    while (i > 0) {
        *buf++ = temp[--i];
    }
    return buf;
}
```

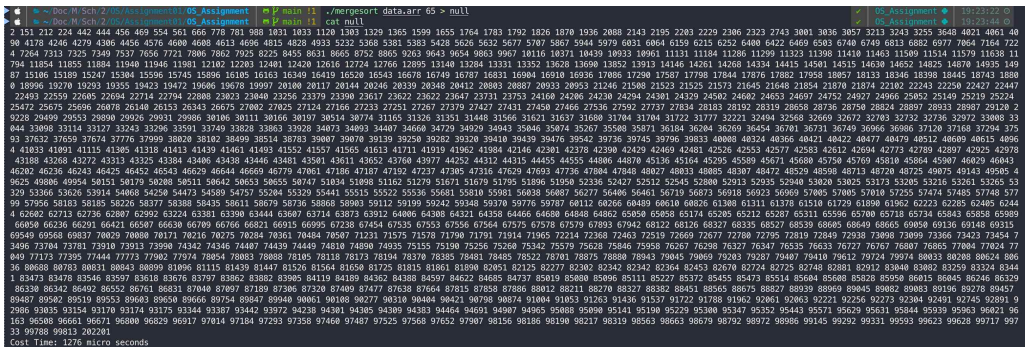
5. 실행결과



실행1 (8Thread 278μs)



실행2 (17Thread 508μs)



실행3 (65Thread 1276μs)

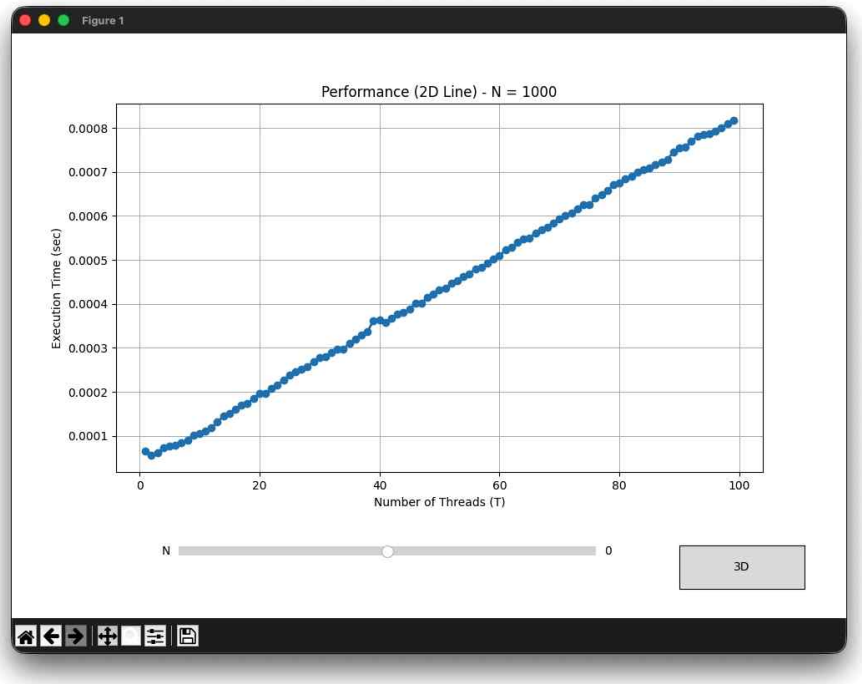


그림 12 스레드수 per time

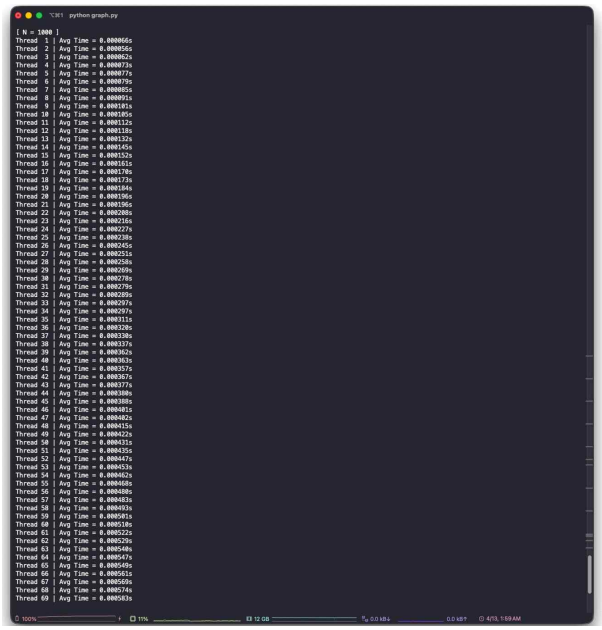


그림 13 실행결과를 파이프라인으로 연결. 최소 시간 56μs

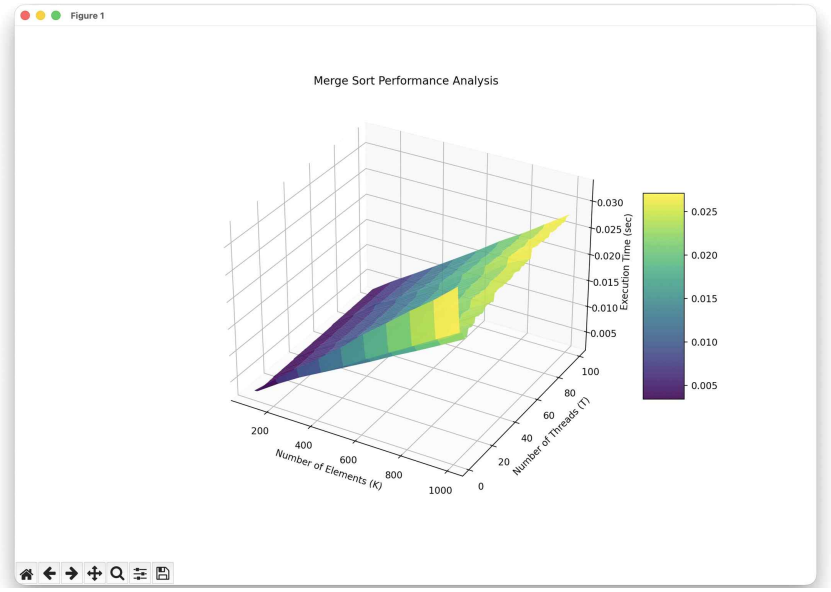
5.1 실험결과 분석

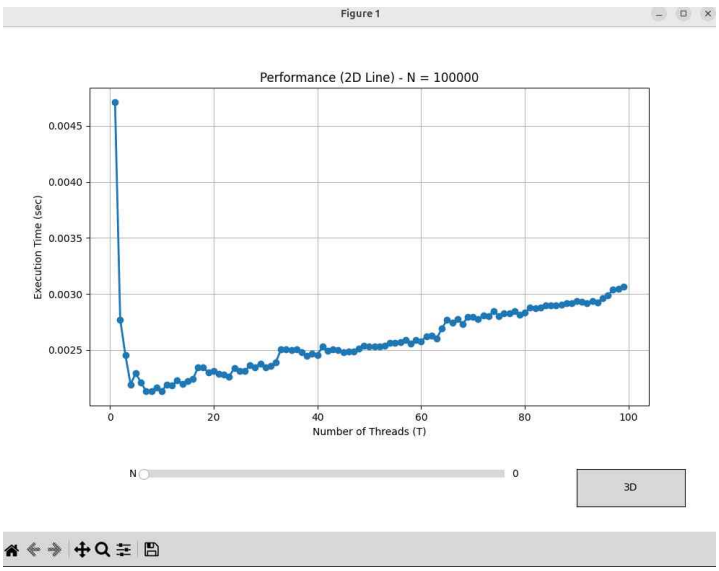
멀티스레드를 활용한 서브셋 정렬은 주어진 배열에서는 pthread를 생성하는 비용자체가 멀티스레드를 활용하므로서 얻는 이점보다 커서 스레드가 많아질 수록 성능이 감소했다.
스레드별 속도를 측정하는 반복작업을 수행하기 위해 만든 python 프로그램에서는 정렬결과를 출력하지 않아 터미널에서의 결과보다 약 4배 빠른 수행속도를 보였다. input.arr 1000여개를 정렬하는데 56μs 라는 매우 준수한 시간 내에 실행된다.

5.2. 추가실험

위 실험결과는 단일스레드가 가장 우수한 성능을 보였다. 멀티스레드의 성능향상을 확인하고자 input이 충분히 많을 때의 성능을 추가로 측정했다.

5.2.1 10만~100만개



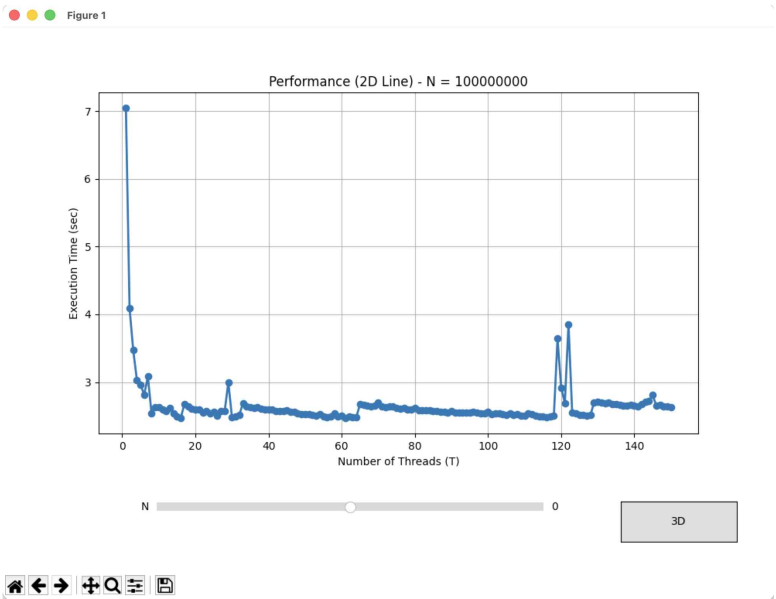


위는 10만~100만개에서 정렬프로그램을 10번씩 수행 한 뒤 각각 평균을 낸 그래프이다. M4칩의 물리적 칩 개수인 10개 부근인 8개에서 가장 뛰어난 성능을(sweet spot) 보였다. 이는 10~100만개일 때 물리적인 코어 수보다 추가적인 스레드를 만드는 비용이 그 이득보다 크다는 사실을 보인다.

추가로 스레드의 수가 2^n+1 일 때 눈에 띄게 성능이 하락하는 패턴을 보였는데 이는 데이터 분할 및 병합 과정의 불균형으로 인한 현상이다. 최종 병합 단계에서 step을 두 배씩 늘려가며 서브셋들을 묶는데, 전체 배열을 끝까지 합치기 위해 도는 Round의 수가 $\log_2 N$의 올림이다. 예를 들어 N이 32일 때는 5라운드 만에 병합이 끝나지만, 33일 때는 6라운드를 돌아야하고, 이는 낭비가 심하다.

이는 M4칩의 물리적 코어 개수인 10개보다 8개에서 실행속도가 더 빠르다는 것을 합리적으로 설명한다.

5.2.2 1억개



본 실험에서는 스레드가 150개까지 많아질 수록 더 빠른 성능을 보였다. 이는 물리적인 코어수를 넘어서 스레드를 만들어도 context switch가 만드는 overhead의 비율이 적기 때문이다. 스레드의 수가 2^n+1 일 때 눈에 띄게 성능이 하락하는 패턴은 여전히 관측되었다. 1억개의 배열을 정렬하는데 약 2초 부근으로 준수한 성능을 보였다.

5.3 실험결과 분석

추가적인 실험을 통해 데이터가 충분히 많아질수록 멀티스레드를 활용한 서브셋 정렬 프로그램이 갖는 이점을 확인할 수 있었다.

6. 결론

본 과제를 수행하면서 멀티스레드 프로그램의 이점과, 컨텍스트 스위칭 비용, pthread오버헤드가 프로그램에 작용하는것을 직관적으로 확인 할 수 있었다.

해당 실험을 Linux에서도 실행해 봤지만, VM특성상 측정 시간 값들이 일정하지 않는 결과가 나왔다. 해당 실험 결과는 github에 첨부한다.

7. 구현 최적화 사항

실행성능을 높이기 위해 다음과 같은 최적화를 적용하였다.

7.1 mmap기반 고속 파일 입력

일반적인 fopen/getline방식 대신 mmap을 사용하여 파일을 읽는다.
mmap은 파일 전체를 메모리에 직접 매핑하여 불필요한 시스템 콜 오버헤드와 버퍼 복사를 방지한다.

7.2 fast_itoa 기반 고속 출력

정렬 결과 출력 시 printf대신 직접 구현한 fast_itoa를 사용한다.
printf는 포맷 문자열 파싱, 내부 버퍼 관리 등의 오버헤드가 존재한다. fast_itoa는 정수를 문자 배열로 직접 변환한 뒤, 하나의 큰 버퍼에 연속으로 기록하고 fwrite로 한 번에 출력함으로써 시스템 콜 횟수를 최소화한다.

8. github

https://github.com/happylife10201020/OS_ThreadMergeSort

github에 올린 코드는 thread pool을 활용해 final merge 또한 병렬로 구현하였다. 과제의 요구사항 명세에 가장 잘 맞는 버전은 8e6ccf8 커밋이다.

자동적으로 배열을 만들어주고, 그래프를 그리는 python 코드는 생성형 AI의 도움을 받아 제작했다. 해당 프로그램들은 uv환경에의 실행을 고려해 제작했다. GUI출력을 위해서 Tkinter가 설치되어있어야 한다.

```
# Download Tkinter (Mac)
brew install python-tk

# 가상 환경 생성
uv venv

# 가상 환경 활성화
source .venv/bin/activate

# 의존성 패키지 설치 (numpy, matplotlib 등)
uv pip install -r requirements.txt
```